

1. Indexing

a) What is Indexing?

An **index** is a database structure used to find rows faster without scanning the entire table.

Think of it like the **index page of a textbook**.

Without an index:

Search "Rashid"



Check row 1

Check row 2

Check row 3

...

Check every row

With an index:

Search "Rashid"



Index



Find location of Rashid



Get the row

Example

```
CREATE INDEX idx_employee_name  
ON employees(employee_name);
```

Now:

```
SELECT *  
FROM employees  
WHERE employee_name = 'Rashid';
```

can potentially be much faster on a large table.

b) Creating and Managing Indexes

Create an index

```
CREATE INDEX idx_employee_name  
ON employees(employee_name);
```

Index multiple columns

```
CREATE INDEX idx_emp_dept_salary  
ON employees(department_id, salary);
```

Unique index

```
CREATE UNIQUE INDEX idx_employee_email  
ON employees(email);
```

This prevents duplicate values.

View indexes in PostgreSQL

```
\d employees
```

Delete an index

```
DROP INDEX idx_employee_name;
```

c) Impact of Indexing on Query Performance

Advantages

- Faster **SELECT** queries
- Faster searching using **WHERE**
- Can improve **JOIN** performance
- Can improve **ORDER BY**
- Can improve some **GROUP BY** operations

Disadvantages

Indexes are **not free**.

They:

- Consume storage

- Make **INSERT** slower
- Make **UPDATE** slower
- Make **DELETE** slower

Why?

Because whenever table data changes, the database may also need to update the index.

Important point for exams

Indexes improve read/search performance but can reduce write performance and consume additional storage.

Don't create an index on **every column**.

2. Stored Procedures

a) Introduction to Stored Procedures

A **stored procedure** is a group of SQL statements stored inside the database that can be executed whenever needed.

Instead of repeatedly writing:

```
UPDATE employees  
SET salary = salary + 5000  
WHERE employee_id = 10;
```

you can create a procedure.

b) Writing and Executing Stored Procedures

PostgreSQL example:

```
CREATE OR REPLACE PROCEDURE increase_salary(  
    emp_id INT,  
    amount INT  
)
```

```
LANGUAGE SQL
AS $$
    UPDATE employees
    SET salary = salary + amount
    WHERE employee_id = emp_id;
$$;
```

Execute it using:

```
CALL increase_salary(10, 5000);
```

Why use procedures?

- Reuse SQL logic
 - Reduce repeated code
 - Perform database operations
 - Can contain multiple SQL statements
 - Useful for business operations
-

3. Functions

a) Introduction to Functions

A **function** is a stored program that performs an operation and **returns a value**.

The key difference you should remember:

Procedure	Function
Called using CALL	Usually called using SELECT
Mainly performs an operation	Returns a value
Doesn't have to return a value	Must have a return type
CALL procedure_name()	SELECT function_name()

b) Writing and Executing Functions

Example:

```
CREATE OR REPLACE FUNCTION get_salary(emp_id INT)
RETURNS INT
LANGUAGE SQL
AS $$
    SELECT salary
    FROM employees
    WHERE employee_id = emp_id;
$$;
```

Execute:

```
SELECT get_salary(10);
```

Another example:

```
CREATE OR REPLACE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
LANGUAGE SQL
AS $$
    SELECT a + b;
$$;
```

Execute:

```
SELECT add_numbers(10, 20);
```

Result:

30

Easy way to remember

Procedure → DO something

Function → RETURN something

4. Transactions

a) Understanding Transactions

A **transaction** is a group of database operations treated as **one unit of work**.

Example: transferring ₹5,000 from Account A to Account B.

There are two operations:

A → -5000

B → +5000

Both should happen.

You don't want:

A → -5000

B → ❌ failed

Otherwise money disappears.

So we use a transaction.

5. ACID Properties

ACID describes the important properties of reliable transactions.

A — Atomicity

All or nothing.

Either every operation succeeds or none of them happen.

Transfer:

A - ₹5000

B + ₹5000

Both succeed → COMMIT

One fails → ROLLBACK

C — Consistency

The database must remain in a **valid state** before and after the transaction.

For example, if a database has a rule:

salary > 0

a transaction shouldn't leave the database with:

salary = -5000

I — Isolation

Multiple transactions happening at the same time should not improperly interfere with each other.

Example:

Transaction A

Transaction B

They may run concurrently, but the database manages their interaction.

D — Durability

Once a transaction is committed, the changes are **permanent**, even if the system crashes afterward.

COMMIT

↓

Data permanently saved

Easy memory trick

All Cats In Dorms

- **A** → Atomicity
 - **C** → Consistency
 - **I** → Isolation
 - **D** → Durability
-

6. BEGIN, COMMIT and ROLLBACK

BEGIN

Starts a transaction.

BEGIN;

Then perform operations:

```
UPDATE employees  
SET salary = salary + 5000  
WHERE employee_id = 10;
```

If everything is correct:

COMMIT;

Changes become permanent.

ROLLBACK

If something goes wrong:

ROLLBACK;

The changes made during the transaction are undone.

Complete example

BEGIN;

```
UPDATE accounts  
SET balance = balance - 5000  
WHERE account_id = 1;
```

```
UPDATE accounts  
SET balance = balance + 5000  
WHERE account_id = 2;
```

COMMIT;

If there is an error:

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 5000  
WHERE account_id = 1;
```

```
-- Something went wrong
```

```
ROLLBACK;
```

7. Concurrency and Deadlocks

What is concurrency?

Concurrency means multiple transactions are executing at the same time.

Example:

Transaction A → Employee 1
Transaction B → Employee 2

The database needs to control them so that they don't corrupt each other's data.

What is a deadlock?

A **deadlock** happens when two transactions are waiting for each other indefinitely.

Example:

Transaction A
↓
Locks Row 1
↓
Waiting for Row 2

Transaction B
↓

Locks Row 2



Waiting for Row 1

Now:

A waits for B

B waits for A

Neither can continue.

That's a **deadlock**.

How to avoid deadlocks?

- Access tables/rows in a consistent order
 - Keep transactions short
 - Avoid unnecessary locks
 - Commit/rollback transactions promptly
 - Avoid holding locks while doing unrelated work
-

8. Normalization

a) Importance of Normalization

Normalization is the process of organizing database tables to reduce:

- Data redundancy
- Duplicate data
- Update problems
- Insert problems
- Delete problems

Bad design

Suppose we have:

student_id	student_name	course	teacher
1	Rashid	Python	Ahmed

2	Ali	Python	Ahmed
3	John	SQL	Ibrahim

Python and Ahmed are repeated.

If there are thousands of students, this creates unnecessary duplication.

Normalization separates the data into related tables.

9. 1NF — First Normal Form

A table is in **1NF** when:

1. Each column contains **atomic/single values**
2. There are no repeating groups

✗ Not 1NF

student_id	name	courses
1	Rashid	Python, SQL

courses contains multiple values.

✓ 1NF

student_id	name	course
1	Rashid	Python
1	Rashid	SQL

Each cell contains one value.

10. 2NF — Second Normal Form

A table must:

1. Already be in **1NF**
2. Have **no partial dependency**

This mainly matters when there is a **composite primary key**.

Example:

(student_id, course_id)

Suppose:

student_id	course_id	student_name	course_name
d	d	e	e

Primary key:

(student_id, course_id)

But:

student_id → student_name

course_id → course_name

These depend on only **part** of the composite key.

That's a **partial dependency**.

Fix

Create separate tables:

Students

student_id

student_name

Courses

course_id

course_name

Enrollments

student_id
course_id

Now the design satisfies 2NF.

11. 3NF — Third Normal Form

A table must:

1. Be in **2NF**
2. Have **no transitive dependency**

Example:

employee_id	employee_name	department_id	department_name
-------------	---------------	---------------	-----------------

We have:

employee_id → department_id
department_id → department_name

Therefore:

employee_id → department_name

indirectly.

That's a **transitive dependency**.

Fix

Employees:

employee_id
employee_name
department_id

Departments:

department_id
department_name

Now department information is stored separately.

12. BCNF

BCNF = Boyce-Codd Normal Form

BCNF is a stronger version of 3NF.

The basic rule is:

Every determinant must be a candidate key.

In simpler terms, if:

$A \rightarrow B$

then **A** should be a candidate key.

For most beginner SQL reviews, remember:

1NF $\rightarrow$ Atomic values

2NF $\rightarrow$ Remove partial dependency

3NF $\rightarrow$ Remove transitive dependency

BCNF $\rightarrow$ Every determinant is a candidate key

13. Normalization — Quick Example

Imagine this table:

Student

student_id

student_name

course_id

course_name

teacher_id

teacher_name

You can identify the dependencies:

student_id → student_name

course_id → course_name, teacher_id

teacher_id → teacher_name

A normalized design could be:

Students

student_id
student_name

Courses

course_id
course_name
teacher_id

Teachers

teacher_id
teacher_name

Enrollment

student_id
course_id

This reduces duplication and makes the database easier to maintain.

What you should be able to answer in a review

Indexing

Q: What is an index?

An index is a database structure that improves the speed of data retrieval.

Q: Disadvantage of indexes?

They consume storage and can slow down INSERT, UPDATE and DELETE operations.

Procedures

Q: What is a stored procedure?

A stored procedure is a group of SQL statements stored in the database and executed when required.

```
CALL procedure_name();
```

Functions

Q: What is a function?

A function is a stored program that performs an operation and returns a value.

```
SELECT function_name();
```

Transactions

Q: What is a transaction?

A transaction is a group of database operations treated as a single unit of work.

```
BEGIN;  
-- operations  
COMMIT;
```

or:

```
BEGIN;  
-- operations  
ROLLBACK;
```

ACID

A → Atomicity → All or nothing

C → Consistency → Valid database state

I → Isolation → Transactions don't improperly interfere

D → Durability → Committed changes are permanent

Normalization

1NF → Atomic values

2NF → No partial dependency

3NF → No transitive dependency

BCNF → Every determinant is a candidate key

The most important thing for your review is being able to look at a table and identify: primary key → dependencies → redundancy → which normal form is violated → how to split the tables.

Database Design & ER Modeling

a) Principles of Good Database Design

Good database design means organizing data so that it is accurate, efficient, consistent, and easy to maintain.

1. Avoid data duplication

Don't store the same information unnecessarily.

✗ Bad:

student_id	student_name	course	teacher
1	Rashid	Python	Ahmed
2	Ali	Python	Ahmed

3	John	Python	Ahmed
---	------	--------	-------

Python and Ahmed are repeated.

Instead, separate them into related tables.

2. Use appropriate primary keys

Every important table should have a way to uniquely identify each row.

`student_id INT PRIMARY KEY`

Example:

student_id	name
------------	------

1	Rashid
---	--------

2	Ali
---	-----

`student_id` uniquely identifies each student.

3. Use foreign keys

Foreign keys establish relationships between tables.

`CREATE TABLE courses (`

`course_id INT PRIMARY KEY,`

`course_name VARCHAR(50)`

);

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course_id INT REFERENCES courses(course_id)  
);
```

Here:

`courses.course_id`

↑

|

`students.course_id`

4. Normalize the database

Use normalization to reduce:

- Duplicate data
- Update anomalies
- Insert anomalies
- Delete anomalies

Usually aim for 3NF unless there is a specific reason to denormalize.

5. Choose appropriate data types

Don't use **VARCHAR** for everything.

age INT

name VARCHAR(100)

salary DECIMAL(10,2)

date_of_birth DATE

6. Maintain data integrity

Use constraints such as:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

DEFAULT

Example:

age INT CHECK (age >= 18)

This prevents invalid data.

b) Entity-Relationship (ER) Modeling

ER modeling is a way of representing the structure of a database before creating the actual tables.

It mainly contains:

Entity

Attribute

Relationship

1. Entity

An entity is a real-world object about which we store information.

Examples:

Student

Teacher

Course

Employee

Department

Product

Customer

For example:

Student

is an entity.

2. Attribute

An attribute describes an entity.

For **Student**:

Student

|— student_id

|— name

|— age

└— email

Here:

student_id

name

age

email

are attributes.

3. Relationship

A relationship describes how entities are connected.

Example:

Student — enrolls in — Course

A student enrolls in a course.

Another example:

Department — has — Employees

c) Creating ER Diagrams

Let's take a simple Student-Course system.

We have:

STUDENT

student_id

name

email

COURSE

course_id

course_name

Relationship:

Student — Enrolls — Course

Cardinality

Cardinality tells us how many entities can participate in a relationship.

The main types are:

1:1 — One-to-One

One person has one passport.

Person — Passport

1 1

1:N — One-to-Many

One department has many employees.

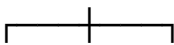
Department — Employee

1 N

One department:

Department 1

↓



↓ ↓ ↓

E1 E2 E3

M:N — Many-to-Many

Students can enroll in many courses, and a course can have many students.

Student — Course

M N

Example:

Rashid → Python

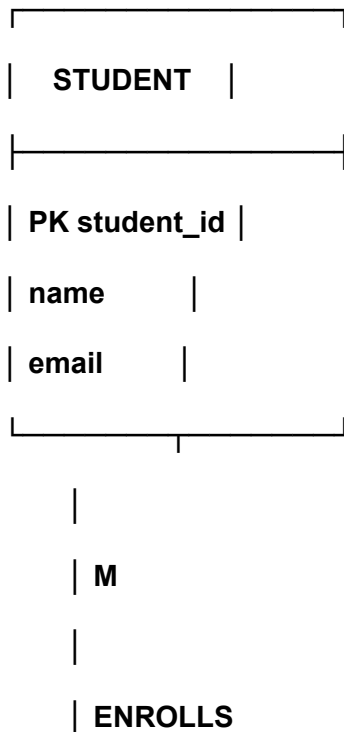
Rashid → SQL

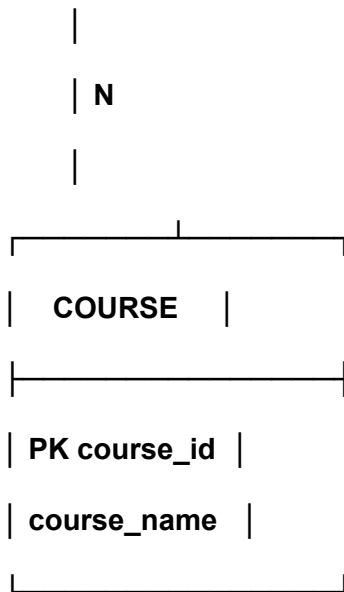
Ali → Python

Ali → Java

ER Diagram Example

For a college database:





This represents:

A student can enroll in many courses, and a course can have many students.

d) Converting ER Diagrams to Tables

This is very important.

The basic rules are:

Rule 1: Entity → Table

ER entity:

STUDENT

becomes:

```

CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
)
  
```

);

Rule 2: Attributes → Columns

ER:

Student

|— student_id

|— name

|— email

becomes:

students

student_id

name

email

Rule 3: Primary key → PRIMARY KEY

If **student_id** uniquely identifies a student:

student_id INT PRIMARY KEY

Rule 4: 1:N → Foreign key on the N side

ER:

Department 1 — N Employee

Tables:

CREATE TABLE departments (

```
department_id INT PRIMARY KEY,  
department_name VARCHAR(100)  
);
```

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100),  
    department_id INT,  
    FOREIGN KEY (department_id)  
        REFERENCES departments(department_id)  
);
```

Why is **department_id** in **employees**?

Because one department has many employees, so the foreign key goes on the many side.

Remember:

1:N → Foreign key goes to N side.

M:N Relationship → Junction Table

This is one of the most important ER-to-table rules.

ER:

Student M ——— N Course

You cannot normally represent M:N directly with just one foreign key.

Create a third table:

STUDENT

student_id

name

COURSE

course_id

course_name

ENROLLMENT

student_id

course_id

SQL:

CREATE TABLE students (

student_id INT PRIMARY KEY,

name VARCHAR(100)

);

CREATE TABLE courses (

course_id INT PRIMARY KEY,

course_name VARCHAR(100)

);

```

CREATE TABLE enrollment (

    student_id INT,

    course_id INT,


    PRIMARY KEY (student_id, course_id),


    FOREIGN KEY (student_id)

        REFERENCES students(student_id),


    FOREIGN KEY (course_id)

        REFERENCES courses(course_id)

);

```

The **enrollment** table is called a:

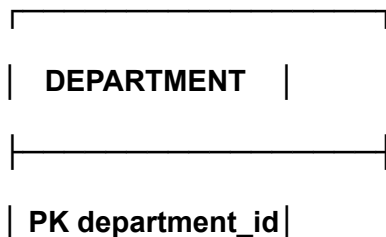
Junction table / Bridge table / Associative table

Complete Example

Suppose the requirement is:

A department has many employees. Each employee belongs to one department.

ER model



```
CREATE TABLE employees (  

    employee_id INT PRIMARY KEY,  

    employee_name VARCHAR(100),  

    salary DECIMAL(10,2),
```

department_id INT,

FOREIGN KEY (department_id)

REFERENCES departments(department_id)

);

So:

ER Diagram



Identify entities



Create tables



Identify attributes



Create columns



Identify primary keys



Identify relationships



Add foreign keys



For M:N → create junction table

Review Questions You Should Know

1. What is an entity?

A real-world object represented in a database.

2. What is an attribute?

A property/characteristic of an entity.

3. What is a relationship?

A connection between entities.

4. What is cardinality?

It describes how many entities participate in a relationship.

5. What are the main cardinalities?

1 : 1

1 : N

M : N

6. In a 1:N relationship, where does the foreign key go?

On the N side.

7. How do you convert an M:N relationship into tables?

Create a junction table containing foreign keys from both entities.

8. What is an ER diagram?

A graphical representation of entities, attributes, and relationships in a database.

Most important thing to memorize

Entity → Table

Attribute → Column

Primary Key → PRIMARY KEY

Relationship → FOREIGN KEY

1 : N → FK on N side

M : N → Junction table